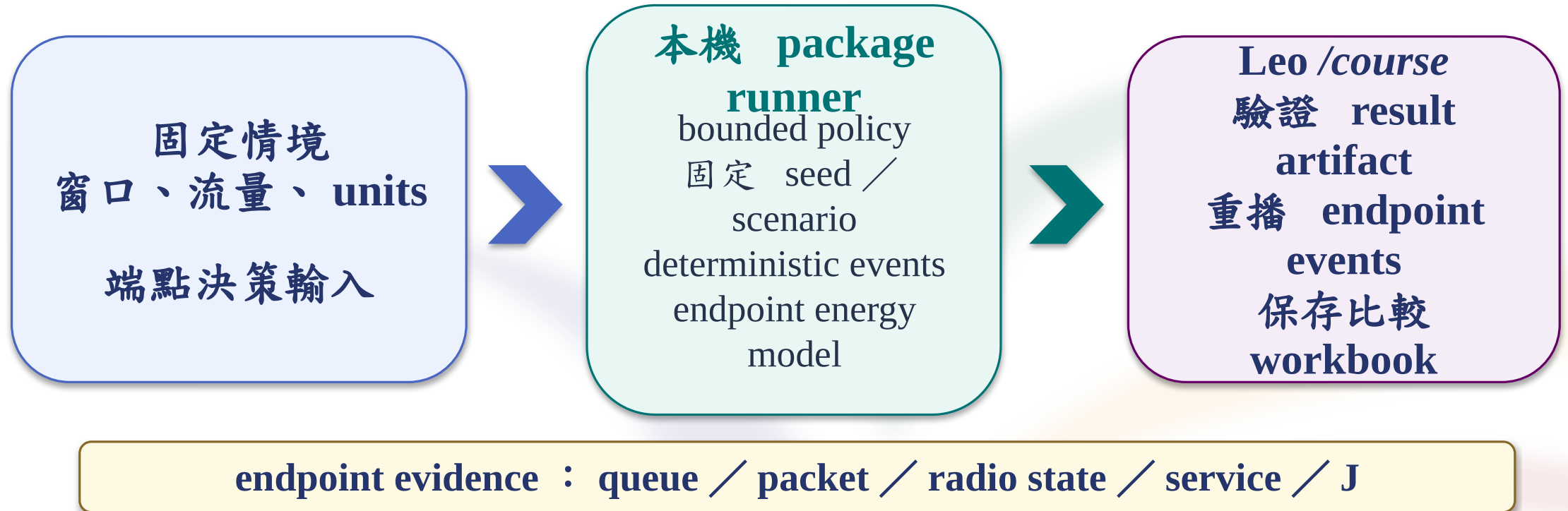


O001 | 智慧節能 IoT：為何使用 LoRaEnergySim

在窗口與期限內完成必要傳輸，同時降低端點能量



scenario_id (情境識別碼) | 來源: fixed scenario package | 作用: 固定窗口、流量與單位
單位: 字串識別碼 | 判讀: 相同情境與相同 policy identity 才進入比較

O002 | 為何加入 LEO：服務窗口會改變

LEO 提供一條會開啟、變強、關閉的服務機會 trace



窗口關閉時，runner 的合法安全分支為休息；窗口開啟時，策略才評估等待或送出。

service_window (服務窗口) | 來源：fixed scenario trace | 作用：定義合法傳輸區間
單位：trace time | 判讀：決定 action 是否具有服務機會

O003 | 封裝已完成什麼；實驗還要做什麼

已固定環境、情境、runner 與 evidence；實驗只改 policy、解釋結果

固定與 setup

- fixed scenario / case
- POSIX / Windows setup scripts
- package-local .venv
- *READY* verify receipt

policy 與 runner

- bounded *student_policy.py*
- legal observation / action API
- deterministic runner / fixed seed
- *result.json* result artifact

evidence 與 recovery

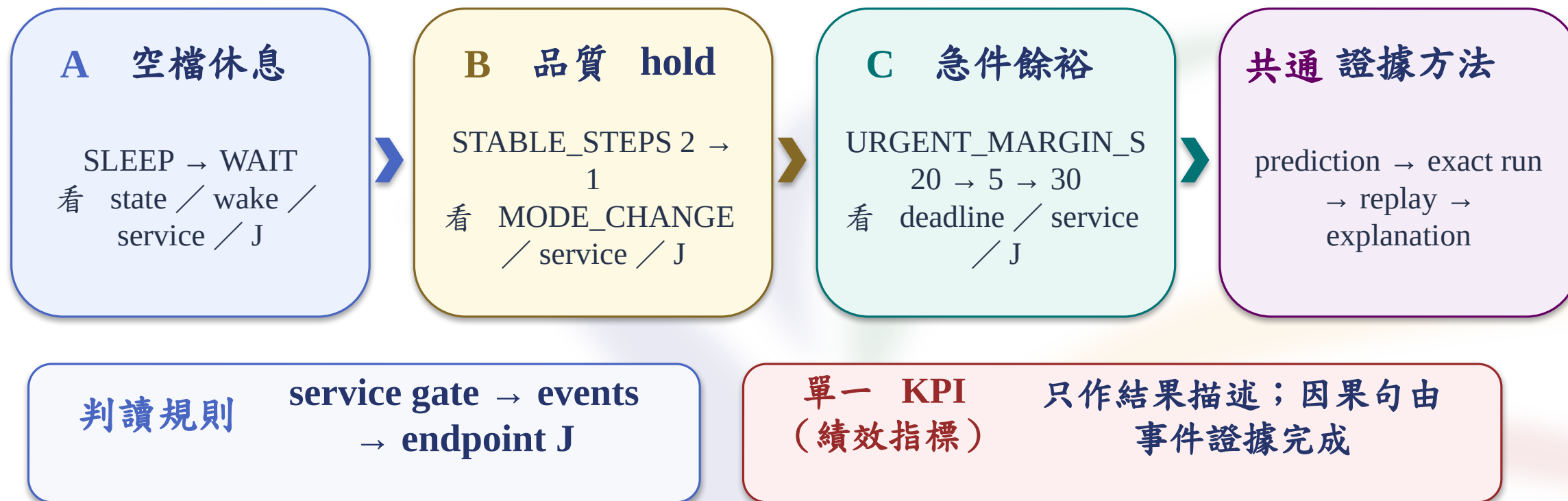
- paired *endpoint-replay.json*
- identity / units / provenance gate
- same-scenario fallback pair
- claim ceiling preserved

實驗操作：改 active marked block → exact run → 上傳 artifact → 解釋 evidence

.venv（套件本地虛擬環境） | 來源：setup script | 作用：隔離 lock、policy、scenario 與 runner
單位：目錄路徑 | 判讀：course.sh / course.cmd 使用同一套 Python 3.11.x

O004 | 三個實驗：改什麼、為何改、看什麼

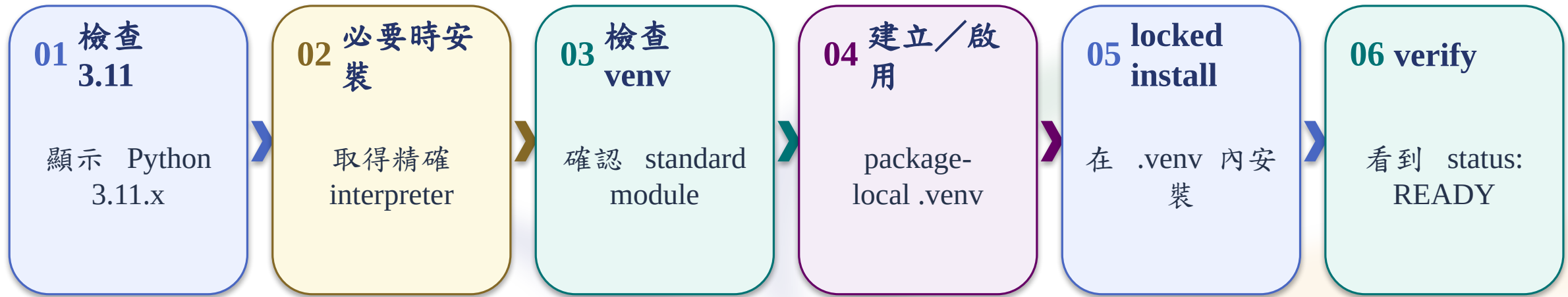
固定 scenario、seed、traffic 與 endpoint scope；每次只改一個控制點



prediction (預測紀錄) | 來源：workbook record | 作用：指定可反駁的變化方向
單位：statement | 判讀：執行前基準與執行後 evidence 對照

O004a | 開始前：環境建立的固定順序

先確認 Python 3.11，再建立隔離環境；READY 之後才開始修改 policy



目的：固定 interpreter 與依賴邊界，讓同一 case 可重跑、可比較

手動流程與 setup script 二選一；不要完成後再重複執行

.venv（套件本地虛擬環境） | 來源：Python venv module | 作用：隔離 interpreter 與 locked requirements

單位：目錄路徑 | 判讀：啟用後 python 必須指向 package root 內的 .venv

O004b | Windows：檢查、建立環境、安裝與驗證

PowerShell 為主要路徑；啟用受阻時改用同一 package 目錄的 Command Prompt

01 | 檢查；缺少才安裝

```
py -3.11 --version

winget install --id=astral-sh.uv -e
uv python install 3.11
```

02 | 建立並啟用 .venv

```
$python311=(uv python find
3.11).Trim()
& $python311 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

03 | 在環境內安裝／verify

```
python -m pip install --require-
hashes
--no-deps -r .\requirements-lock.txt
.\course.cmd verify
```

PowerShell activation 成功
python --version → Python 3.11.x
python path → .venv\Scripts\python.exe

若 Activate.ps1 受阻
Command Prompt：call .venv\Scripts\activate.bat
再執行同一 locked install 與 course.cmd verify

PYTHON_BIN（Python 執行器路徑） | 來源：Python Launcher 或 `uv python find 3.11` | 作用：選定精確 3.11 interpreter
單位：檔案路徑 | 判讀：建立 .venv 前先顯示 Python 3.11.x

O004c | Linux / WSL : 檢查 3.11 , 再建立 .venv

WSL 使用其 Linux distribution ; 命令與 Windows shell 不混用

01 | 檢查 ; 缺少才安裝

```
command -v python3.11
python3.11 --version

curl -LsSf
https://astral.sh/uv/install.sh | sh
uv python install 3.11
```

02 | 建立並啟用 .venv

```
PYTHON_BIN="$(uv python find
3.11)"
"$PYTHON_BIN" -m venv .venv
source .venv/bin/activate
python --version
```

03 | 在環境內安裝 / verify

```
python -m pip install --require-
hashes
--no-deps -r requirements-lock.txt
bash course.sh verify
```

Ubuntu 24.04 / WSL Ubuntu 24.04
預設 repository 不提供 Python 3.11 ; 使用
uv 取得精確版本

若 distribution Python 缺少 venv /
ensurepip
改用 uv-managed Python 建立 .venv

`source .venv/bin/activate` (啟用虛擬環境) | 來源 : POSIX shell | 作用 : 讓後續 `python` / `pip` 指向
package-local environment
單位 : shell state | 判讀 : `python --version` 顯示 Python 3.11.x

O004d | macOS : Homebrew 或 uv 取得 Python 3.11

不要假設系統 `python3` 是 3.11 ；先查版本，再選一條安裝路徑

01 | 檢查／安裝

```
command -v python3.11
python3.11 --version

brew install python@3.11
# 或 : uv python install 3.11
```

02 | 建立並啟用 .venv

```
PYTHON_BIN="$(brew --prefix
python@3.11)/bin/python3.11"
"$PYTHON_BIN" -m venv .venv
source .venv/bin/activate
```

03 | 在環境內安裝／verify

```
python --version
python -m pip install --require-
hashes
--no-deps -r requirements-lock.txt
bash course.sh verify
```

Homebrew 路徑

```
$(brew --prefix python@3.11)/bin/python3.11
```

uv 路徑

```
PYTHON_BIN=$(uv python find 3.11)
```

`python@3.11` (Homebrew Python 套件) | 來源 : Homebrew formula | 作用 : 提供維護中的 Python 3.11 interpreter
單位 : 安裝套件／路徑 | 判讀 : 建立 `.venv` 前先顯示 Python 3.11.x

O004e | Verify 成功：看到 READY 才進入實驗

READY 是環境與契約 gate；不是 Lab 結果，也不是服務成功

```
{
  "status": "READY",
  "python_version": "3.11.x",
  "scenario_id": "ntpu-energy-decision-01",
  "policy_api_version": "lora-energy-policy-v1",
  "engine_mode": "coherent-course-simulated-adapter",
  "upstream_execution": false
}
```

Verify 檢查什麼

Python 3.11 / scenario identity / baseline
markers / lock / policy API / engine mode /
claim boundary

成功後做什麼

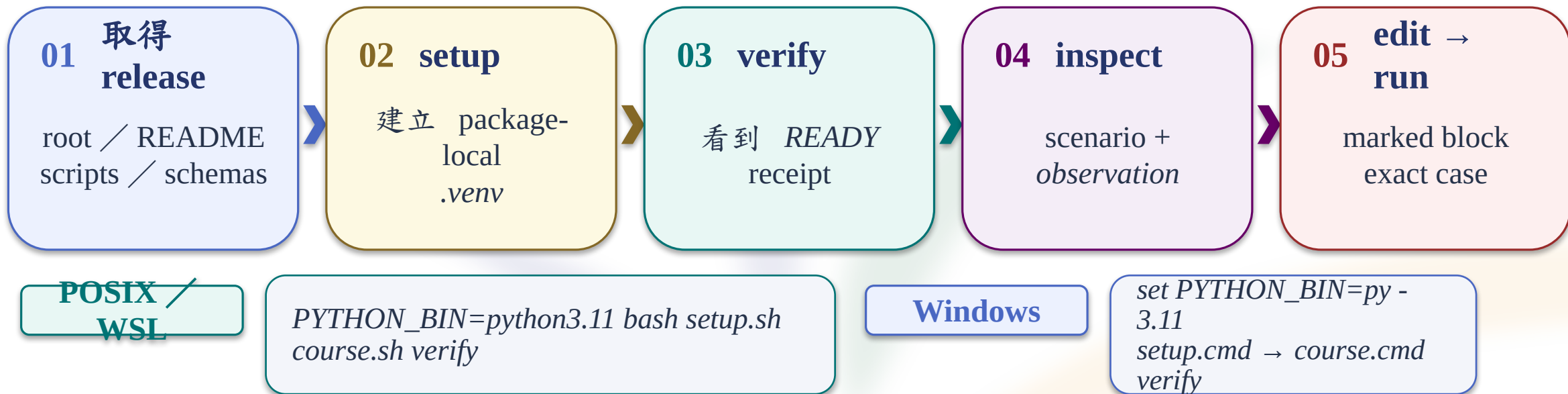
保持 .venv 啟用 → 只改 active marked block
→ exact run
→ 取得 result_path → 上傳 /course

artifacts/verify-receipt.json 只證明環境 READY；Lab 結果由後續 run 產生

status (驗證狀態) | 來源：course.sh / course.cmd verify | 作用：表示環境與固定契約是否一致
單位：enum | 判讀：值為 READY 才進入 case；不等同 service_pass

O005 | release → exact run : 五步操作

先建立可重現環境，再檢查契約，最後才修改 policy 並執行 case

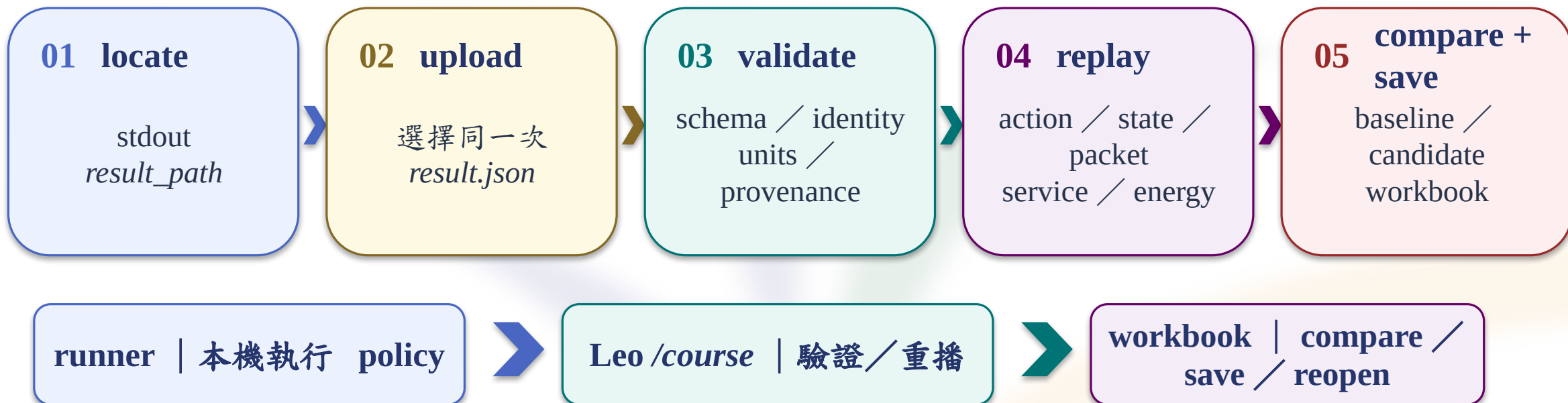


READY 是環境 gate ; result artifact 要由後續 exact run 產生。

READY (環境驗證狀態) | 來源: `course.sh / course.cmd verify` | 作用: 確認 Python、lock、policy API、scenario
單位: status | 判讀: READY 允許進入 runner ; 尚未代表 result artifact 存在

O006 | result.json → /course : 驗證、重播、保存

runner stdout 的 result path 把同一次 run 交給網站處理



result_path (結果路徑) | 來源: runner stdout | 作用: 定位同一次 run 的 result.json 與 replay pair
單位: 檔案路徑 | 判讀: 以 stdout path 開啟; 上傳端點: /course

O007 | student_policy.py : observation → action

讀取允許的 observation，依序判斷，回傳一個 legal action

observation（決策觀測紀錄） | 來源：runner 每個 decision step | 作用：提供合法的 current / past inputs
單位：structured record | 判讀：欄位供 branch 使用，future trace 與 result summary 不進入輸入

```
def choose_action(observation):
    if not observation.contact_open:
        return SLEEP
    if observation.urgent_pending:
        if due_in_s <= URGENT_MARGIN_S:
            return SEND_URGENT
    if queue_size >= BATCH_SIZE:
        return FLUSH_BATCH
    return WAIT
```

欄位／中文	來源	作用	單位	判讀
contact_open (窗口狀態)	scenario trace	合法 action gate	bool	false → SLEEP
urgent_pending (緊急待送)	queue ledger	急件分支	bool	true → 檢查期限
urgent_due_in_s (期限剩餘)	traffic card	比對 margin	s	越小越接近
queue_size (佇列數)	current queue	batch decision	count	≥ batch → FLUSH

legal action : SLEEP / WAIT / SEND_ONE /
SEND_URGENT / FLUSH_BATCH

action（策略動作） | 來源：policy API return | 作用：選擇下一個 state / packet 路徑
單位：enum | 判讀：runner 將 token 展開成可回看的 event ledger

O008 | 可編輯面：active marked block

每次只改一個 marker；其餘 policy 與 course contract 維持凍結

student_policy.py

```
# === LORA EDITABLE: lab-a-pace-rest ===  
PACE_GAP_STEPS = 2  
REST_DURING_GAP = SLEEP
```

```
# === LORA EDITABLE: lab-b-enter-exit-hold  
===  
STABLE_STEPS = 2
```

```
# === LORA EDITABLE: lab-c-batch-urgent  
===  
URGENT_MARGIN_S = 20
```

policy identity（策略識別）| 來源：receipt
作用：連結 edit / predecessor
判讀：hidden / surprise 不變

可編輯：active block

- 一個 bounded constant 或 branch
- WAIT / SLEEP 等 typed symbol
- 一個 causal variable + prediction

保護：course contract

- scenario / quality trace / traffic
- schemas / runner / energy model
- generated result / replay JSON
- Leo source / baseline policy

O009 | exact run 產生可配對的 result 與 replay

同一個 case、同一個 package root、同一次 run identity 產生兩個 JSON artifact

POSIX / WSL

```
bash course.sh run --lab A --case baseline
```

package root → runner → stdout result path

```
{"status": "OK",  
 "result_path": "artifacts/<run_id>/result.json"}
```

Windows

```
course.cmd run --lab A --case baseline
```

package root → runner → stdout result path

同一目錄：endpoint-replay.json
同一 run_id | summary + events + source

result_path 以 stdout 為準；配對 replay 取同一個 generated run directory。

run_id (執行識別碼) | 來源：runner result identity | 作用：配對 result.json 與 endpoint-replay.json
單位：content identity | 判讀：同一次 run 的 path、summary、events 與 replay 必須一致

O010 | /course : artifact → endpoint evidence

網站只處理已產生的 artifact：驗證、重播、ledger 與 workbook

服務 FAIL	端點能量 6.92 J	已送達資料 4800 bit	端點 bit/J 693.641618
------------	----------------	-------------------	------------------------

端點重播 · 選取畫面				1 / 56 · 0s
無線電 SLEEP	動作 —	佇列數 1	累積端點能量 J 0 J	
經過時間 0s	接觸識別碼 —	接觸狀態 關閉	連線品質 0	

實驗／案例	顯示	角色	服務	端點能量 J	來源
A / baseline	目前	基準	FAIL	6.92 J	同情境備用資料

current evidence/course-20260811 · artifact source : same-scenario fallback

01 validate / import

schema、identity、units、provenance

02 endpoint replay

radio、action、queue、contact、quality、J

03 ledger / workbook

service gate、比較、保存與 reopen

endpoint radio / processing ; LEO / system energy 維持獨立

service（服務結果） | 來源：imported result summary | 作用：判斷 required delivery、deadline、freshness
單位：pass / fail | 判讀：網站先呈現服務 gate，再顯示 endpoint energy 與 replay context

O011 | 結果判讀：service → state → endpoint J

同一個 result / replay pair 依服務、事件、端點能量三層完成解釋

01 SERVICE GATE | 服務閘門

required delivery / deadline / freshness →
service_pass

02 PACKET + RADIO | 封包狀態

action → radio_state interval → attempt / retry /
delivery / expiry

03 ENDPOINT ENERGY | 端點能量

state power × time → endpoint_energy_j (J)

endpoint bit/J (端點能量效率)

來源：result summary

單位：bit/J

作用：delivered bits ÷ endpoint J

判讀：service gate 通過後再解讀

evidence boundary

endpoint J ≠ LEO / system / canonical
claim ceiling 隨 artifact 保存

radio_state (無線狀態) | 來源：runner events / endpoint replay | 作用：說明每段 state duration 與 energy bucket

單位：state enum + duration | 判讀：把 action 連回 packet、service 與 endpoint_energy_j

SIMULATED TEACHING DATA / NOT LIVE / NOT MEASURED / NOT CANONICAL-PARITY-VERIFIED